



VISÃO COMPUTACIONAL

AULA 2



Prof. Leonardo Gomes



CONVERSA INICIAL

Nesta aula, discutiremos o conteúdo relacionado a uma etapa muito importante da visão computacional, o pré-processamento. Vamos falar sobre acesso de informações da imagem, histograma de cores, ruído em imagens, entre outros, com exemplos práticos e códigos em python e opencv.

Ao final desta aula, esperamos atingir os seguintes objetivos que serão avaliados ao longo de nossos estudos de forma indicada.

Objetivos
1 – Entender a causa dos ruídos nas imagens e possíveis soluções algorítmicas
2 – Calcular histograma de imagens e gerar gráficos
3 – Aplicar transformações geométricas sobre as imagens.

TEMA 1 – ACESSANDO A IMAGEM

O pré-processamento é uma das etapas principais dentro da sequência de atividades desempenhada em uma solução de visão computacional habitual. Para essa sequência de etapas, dá-se o nome de **pipeline**, termo que vem do inglês para descrever atividades de sequências que se assemelham a uma linha de produção.

O pré-processamento tem por objetivo preparar a imagem para as etapas seguintes, seja atuando defeitos na imagem ou ressaltando características relevantes da **região de interesse** da imagem.

E tudo isso começa com o básico: como acessar a informação de cor de cada pixel e como acessar as informações gerais da imagem, como as dimensões da imagem, quantidade de canais de cores, entre outras informações.



1.1 Acessando pixels

Como já discutimos, uma imagem pode ser entendida como uma matriz com diversas linhas e colunas, e cada célula dessa matriz carrega os valores das cores, RGB do pixel ou uma informação única que é o tom de cinza.

O código a seguir demonstra como carregar uma imagem chamada mario.jpg e, na sequência, acessar o valor das cores RGB na linha 100 coluna 200 dessa imagem.

```
import cv2
imagem = cv2.imread("mario.jpg")
valorPixel = imagem[100,200]
print(valorPixel)
```

Saída do programa:

[0 4 9]

Ao acessar a posição da imagem, recebemos como retorno uma lista com valores de vermelho – 0, verde – 4 e azul – 9 (RGB), nessa ordem. O opencv é bastante flexível, permite inclusive acessar diretamente cada canal de cor, como uma estrutura de dados tridimensional.

```
import cv2
imagem = cv2.imread("mario.jpg")
valorVermelho = imagem[100,200,0]
valorVerde = imagem[100,200,1]
valorAzul = imagem[100,200,2]
print(valorVermelho,valorVerde,valorAzul)
```

Saída do programa:

0 4 9

Caso a imagem seja tons de cinza, o retorno seria um valor único, como no exemplo a seguir.

```
import cv2
```



```
imagem = cv2.imread("mario.jpg")
imagem = cv2.cvtColor(imagem, cv2.COLOR_RGB2GRAY)
valorPixel = imagem[100,200]
print(valorPixel)
```

Saída do programa:

6

Na terceira linha de código, a imagem colorida está sendo convertida para tons de cinza e sobrescrevendo a imagem original. O retorno do programa agora é um único valor, 6.

Se ao invés de ler desejarmos modificar um pixel, então basta atribuir o novo valor ao pixel selecionado como no código a seguir.

```
import cv2
imagem = cv2.imread("mario.jpg")
imagem[100,200] = [50,100,150]
imagem[100,200,2] = 250
valorPixel = imagem[100,200]
print(valorPixel)
```

Saída do programa:

[50 100 250]

Observe que, na linha 3, o código de todos os elementos é modificado simultaneamente, enquanto, na linha 4, apenas o azul.

1.2 Acessando informações da imagem

Ao utilizarmos o método `imread`, o retorno será um objeto da classe `Mat`, que é a estrutura padrão para trabalhar com imagens no `opencv`. Essa classe possui diversos métodos e atributos. Se desejarmos analisar a imagem como um todo, possuímos dois atributos importantes: *shape* e *size*. Vamos observar o código a seguir.

```
import cv2
```



```
imagem = cv2.imread("mario.jpg")
print(imagem.shape) # linhas colunas cores
print(imagem.size) # tamanho
print(imagem.size/imagem.shape[2]) #total pixels
print(imagem.shape[0]*imagem.shape[1])#total pixels
```

Saída do programa:

(343, 250, 3)

257250

85750.0

85750

O atributo *shape* retorna uma tupla com as informações de número de linhas, colunas e canais de cores, nessa ordem. Enquanto o atributo *size* retorna o tamanho da estrutura em questão, ou seja, a multiplicação das suas dimensões, linhas, colunas e canais de cor.

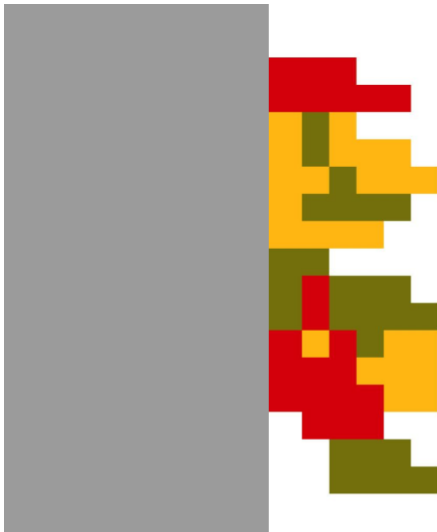
Para chegar no total de *pixels*, é possível dividir o valor de *size* pela quantidade de canais ou multiplicar linhas por colunas, conforme é demonstrado nas últimas duas linhas do código anterior.

```
import cv2
imagem = cv2.imread("mario.jpg")
linhas = imagem.shape[0]
colunas = imagem.shape[1]
imagem[0:linhas,0:colunas//2] = [155,155,155]
#A linha acima é equivalente ao código abaixo
#for i in range(linhas) :
#    for j in range(colunas//2) :
#        imagem[i,j]= [155,155,155]
cv2.imshow('minha imagem',imagem)
cv2.waitKey(0)
```

No código anterior, passamos pelas coordenadas da metade esquerda da imagem, pintando a mesma de cinza. A seguir, a Figura 1 ilustra o resultado.



Figura 1 – Exemplo de acesso aos *pixels*



Fonte: Gomes, 2021

TEMA 2 – HISTOGRAMA DE CORES

O histograma é uma ferramenta importante na análise de dados, geralmente é representado com um gráfico que ilustra a quantidade de ocorrências de determinados dados, por exemplo, um gráfico que ilustra com barras o total de vendas de uma loja em cada mês é um histograma. No caso do histograma de cores, analisamos a frequência de ocorrência de cada cor na imagem. Esse histograma é capaz de nos fornecer informações importantes para corrigir uma série de eventuais problemas com a imagem.

No código a seguir, demonstramos a leitura de uma imagem, o parâmetro 0, no método *imread* força a leitura em tons de cinza. Na sequência, ela é transformada em preto e branco pelo método *threshold*, o que for abaixo de 127 será preto, 0 e acima será branco 255. Na sequência, cada *pixel* é visitado somado no agregador apropriado de branco ou preto.

```
import cv2
#Parametro 0 carrega a imagem em tons de cinza
img = cv2.imread("mario.jpg",0)
linhas = img.shape[0]
colunas = img.shape[1]
#transformando a imagem em binaria
```



```
#abaixo de 127 (segundo parametro) zero
#do contrario sera 255 (terceiro parametro)
#Retorno: limite (lm), no caso 127, e a imagem
lm,img=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
branco=0
preto=0
for i in range(linhas):
    for j in range(colunas):
        if img[i,j] == 255:
            branco+=1
        else:
            preto+=1
print('Total branco:',branco)
print('Total preto:',preto)
```

Saída do programa:

Total branco: 776066

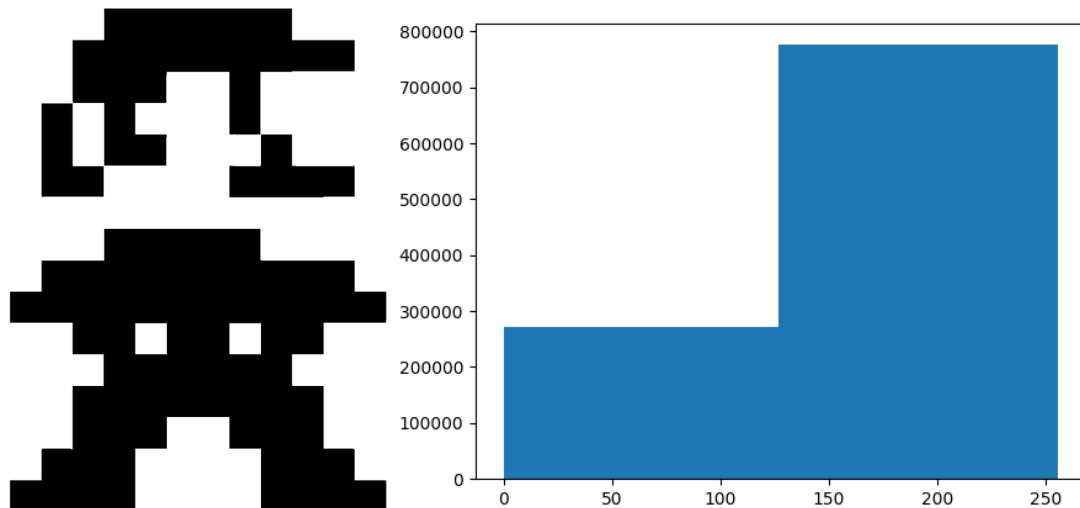
Total preto: 272510

A biblioteca `matplotlib` também possui método próprio para calcular histogramas que gera um gráfico, facilitando ainda mais a análise. No código a seguir, o método `hist` é chamado. No primeiro parâmetro, a imagem é contida para uma lista com o método `ravel()`, o parâmetro seguinte é o que chamamos de *bins*, que são os intervalos do histograma, o que for de 0 até 127 estará no primeiro *bin*, e o que estiver no intervalo de 127 até 256 estará no segundo *bin*.

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("mario.jpg",0)
lm,img=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
plt.hist(img.ravel(), [0,127,256])
plt.show()
```



Figura 2 – Imagem binária e seu histograma respectivo ao lado



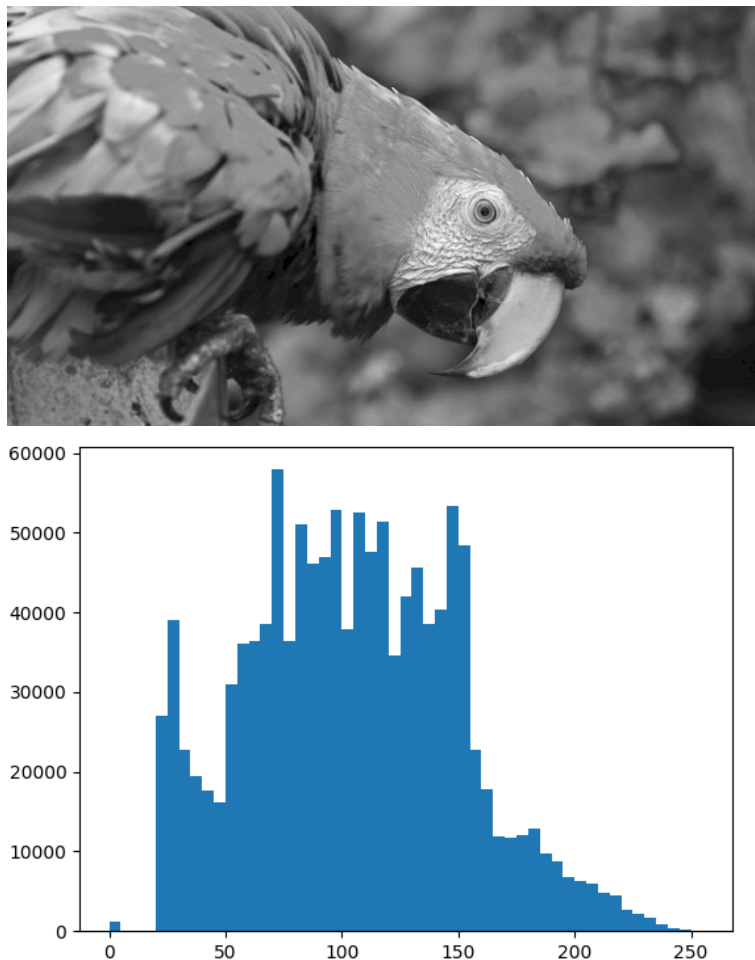
Na Figura 2, é possível verificar, ao analisar o histograma, que existem aproximadamente 3 vezes mais *pixels* brancos do que pretos. Esse tipo de informação pode ser pertinente para identificar, por exemplo, se a iluminação da imagem está adequada.

Ao analisarmos o histograma de uma imagem em tons de cinza, podemos tirar conclusões similares, como a seguir o código, imagem e histograma gerados. No código a seguir, para gerar os *bins*, estamos considerando do 0 até 255 em intervalos de 5 em 5.

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("arara.png",0)
plt.hist(img.ravel(), range(0,256,5))
plt.show()
```




Figura 3 – Imagem tons de cinza e seu histograma respectivo



Crédito: Goran Jakus/Shutterstock.

Observando o histograma da Figura 3, identificamos que a grande maioria dos *pixels* fica entre 20 e 200, um intervalo relativamente largo que caracteriza uma imagem com bom contraste.

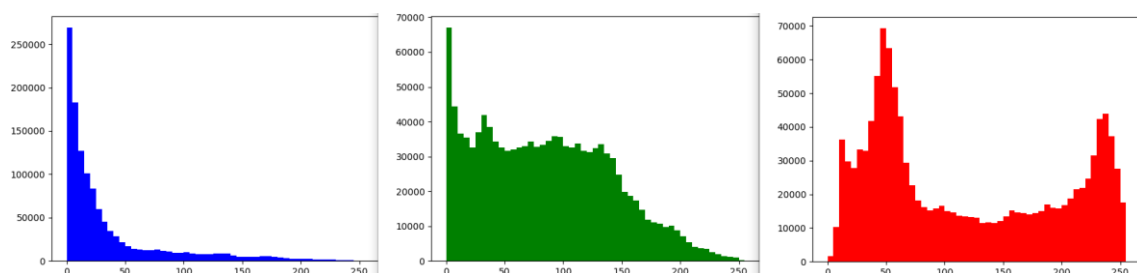
É possível fazer o mesmo com cada canal de cor de uma imagem RGB, confira o código, imagem e histograma a seguir.

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("arara.png")
azul, verde, vermelho = cv2.split(img)
plt.hist(azul.ravel(), range(0,256,5),color='blue')
plt.figure()
```



```
plt.hist(verde.ravel(),  
range(0,256,5),color='green')  
plt.figure()  
plt.hist(vermelho.ravel(),  
range(0,256,5),color='red')  
plt.show()
```

Figura 4 – Imagem colorida e histograma respectivo de cada canal de cor, respectivamente, azul, verde e vermelho



Crédito: Goran Jakus/Shutterstock.

Na Figura anterior, vemos que cada histograma de cada canal de cor conta uma história bem diferente. A imagem é predominantemente vermelha, com bastante verde e pouco azul, por isso os *pixels* azuis estão concentrados em valores próximos ao zero, enquanto os demais estão melhor distribuídos e o vermelho com um pico próximo do valor 250.



2.1 Equalização de histograma

O objetivo de gerarmos histograma é justamente o de analisarmos a distribuição das cores, por exemplo, imagens muito claras, também chamadas de superexpostas, vão concentrar os *pixels* em uma pequena faixa de valor de intensidade luminosa alta. O mesmo ocorre com imagens muito escuras, também chamadas de subexpostas, que concentram os *pixels* em intensidades luminosas baixas. E isso tende a significar uma imagem com problemas de nitidez, que, muitas vezes, afeta a qualidade da leitura da região de interesse da imagem.

Sendo assim, a ideia da equalização do histograma é modificar a cor dos *pixels* de forma que ocupem uma faixa maior de valores, aumentando, assim, a nitidez da imagem. O método que realiza essa tarefa está presente na biblioteca *opencv* com o nome de *equalizeHist*. No código a seguir, veja um exemplo com alguns resultados para ilustrar a discussão.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("caracol.png", 0)
resultado = cv2.equalizeHist(img)
cv2.imshow("Original", img)
cv2.imshow("Equalizada", resultado)
plt.hist(img.ravel(), range(0,256,5))
plt.figure()
plt.hist(resultado.ravel(), range(0,256,5))
plt.show()
```

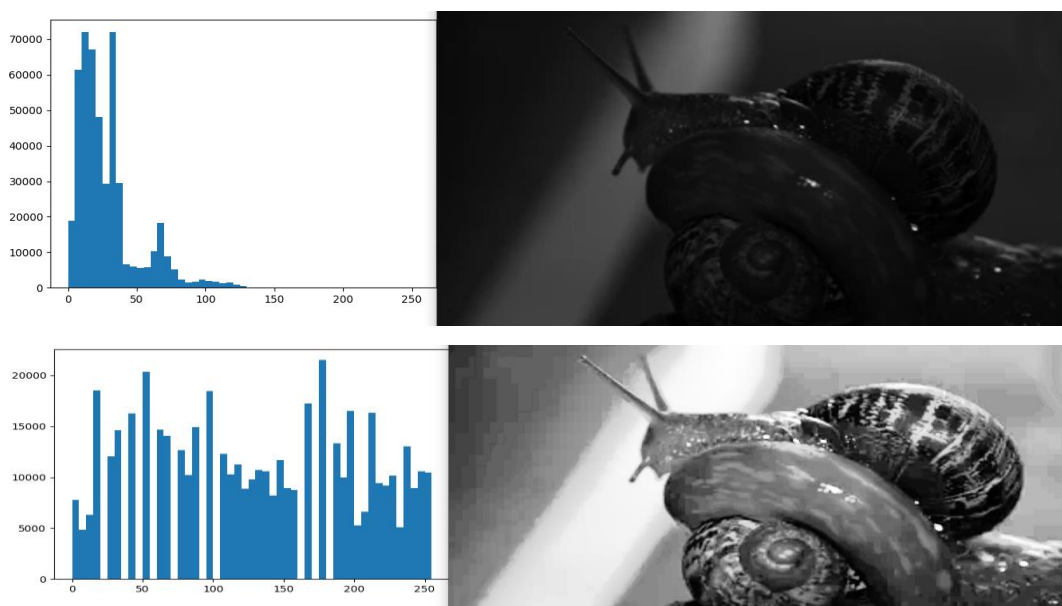
A seguir, nas figuras 5 e 6, temos os resultados da execução do código anterior. Na Figura 5, vemos que a imagem original está subexposta com pouca nitidez, se desejarmos, por exemplo, desenvolver um programa para contabilizar a proporção de manchas no corpo do caracol, como uma medida para categorizar sua espécie, a imagem equalizada facilitaria muito esse processo, pois na imagem original mal distingue-se a textura do corpo do animal.



Vejamos também os histogramas da Figura 5, os *pixels* estão quase que em sua totalidade concentrados no intervalo de 0 até 100, enquanto uma vez equalizado, os pixels estão distribuídos por todas as intensidades luminosas.

Na Figura 6, vemos uma situação similar, agora com uma imagem superexposta. Pelo histograma, os *pixels* da imagem original estão quase todos concentrados no intervalo de 200 até 255 e, depois de equalizada, identificamos melhor os contornos da imagem.

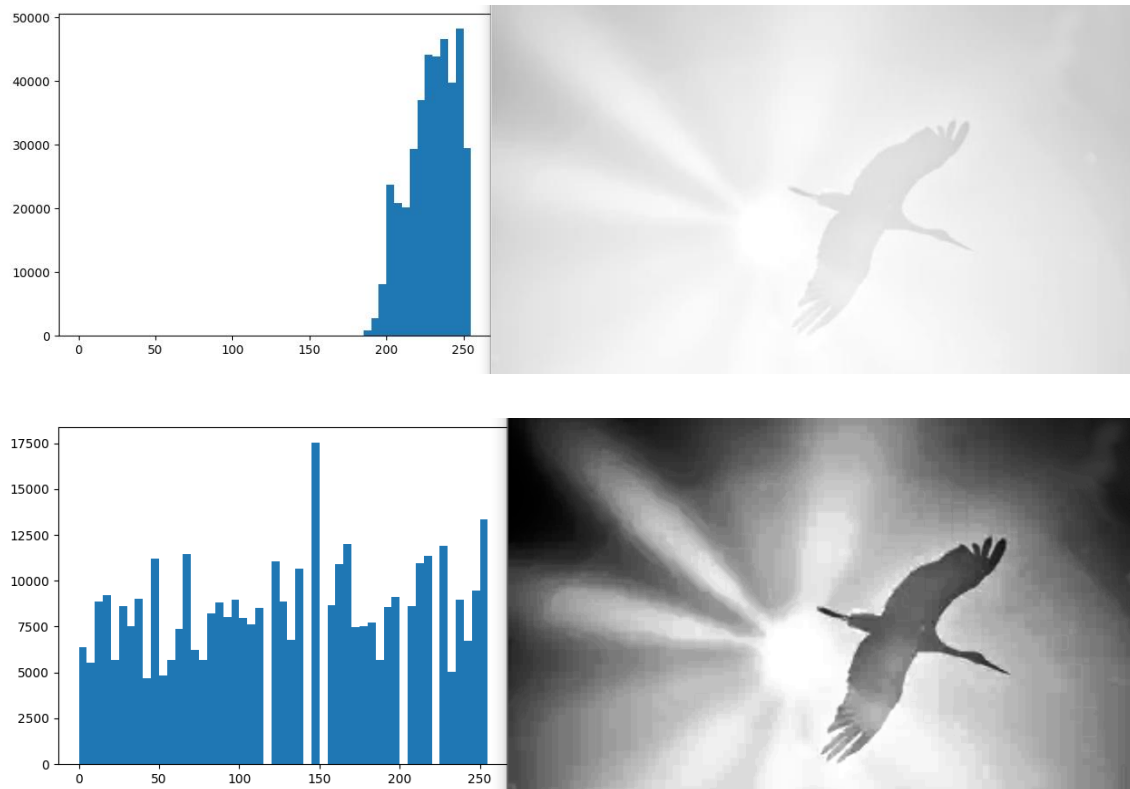
Figura 5 – Equalização da imagem subexposta de um caracol. Histograma e imagem original acima e histograma equalizador e imagem equalizada a seguir



Crédito: 4-life-2-b/Shutterstock.



Figura 6 – Equalização da imagem superexposta de uma gaivota. Histograma e imagem original anterior e histograma equalizador e imagem equalizada a seguir



Crédito: ezp/Shutterstock.

Nos exemplos anteriores, vemos o código sendo executado com imagens em tons de cinza. É possível também equalizar imagens colorizadas. No entanto, em vez de equalizarmos cada canal de cor RGB individualmente, o que desconfiguraria as características das cores da imagem, podemos converter a imagem do espaço de cor RGB para o HSV e utilizar o canal V de intensidade luminosa para equalizar a imagem e, depois, converter uma segunda vez para o RGB novamente, assim, teremos um resultado visualmente coerente.

TEMA 3 – TRANSFORMAÇÕES GEOMÉTRICAS EM IMAGENS

Neste tema, vamos debater as transformações geométricas nas imagens, que são as diferentes formas de transformar imagens de um plano geométrico para outro. Entre as operações mais importantes que veremos aqui, destacamos:

- Rotação: consiste em girar a imagem partindo de um eixo específico.
- Translação: que seria mover a imagem horizontalmente e verticalmente.



- Escala: que seria aumentar ou diminuir a imagem nos eixos horizontal e vertical.
- Perspectiva: uma combinação das outras transformações para dar a impressão de que a imagem está sendo observada de outro ponto de vista.

As imagens são representadas como matrizes bidimensionais tais quais a da Matemática e, sendo assim, as operações matriciais que são estudadas no campo da Geometria Analítica se aplicam aqui da mesma forma. Como os computadores não passam de máquinas avançadas de calcular, essa forma de trabalhar com as imagens como matrizes é executada com excelente desempenho, especialmente nas placas de vídeo que possuem *hardware* focado para trabalhos assim.

3.1 Rotação

A rotação na biblioteca opencv é feita por meio de duas funções principais: *getRotationMatrix2D* e *warpAffine*. Com a função *getRotationMatrix2D*, obtemos a matriz bidimensional capaz de gerar a transformação. Ela precisa de 3 parâmetros: o eixo no qual a rotação será executada; ângulo da rotação em graus; e um fator de escala.

Quanto ao eixo de rotação, imagine a imagem como uma folha de papel solta na mesa, você fixa a folha pressionando ela com um único dedo e, então, gira a folha, o ponto em que seu dedo está fixado é o eixo de rotação dessa operação. Idealmente, esse ponto deve ser o centro da imagem. Ou seja, o total de linhas dividido por 2 e o total de colunas também dividido por 2.

O segundo parâmetro é o ângulo da rotação simplesmente dado em graus, valores positivos indicam rotação no sentido anti-horário. E o terceiro parâmetro é a escala, que deve ser 1 para quando não se deseja modificar as dimensões da imagem. A linha de código a seguir gera uma matriz para rotacionar imagem em 90 graus no sentido anti-horário.

```
matriz = cv2.getRotationMatrix2D(
(colunas / 2, linhas / 2), 90, 1)
```



Para aplicarmos uma matriz de rotação em uma imagem, executamos a linha a seguir, os parâmetros são a imagem Original, a matriz de transformação e as novas dimensões da imagem:

```
imgNova =  
cv2.warpAffine(imgOrig, matriz,(colunas,linhas))
```

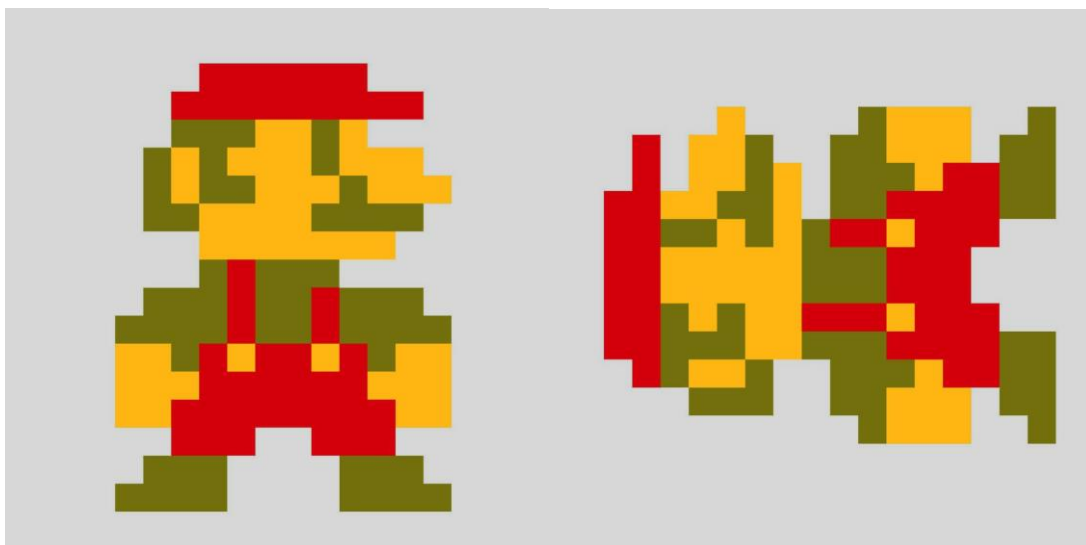
Detalhes das operações matemáticas envolvidas são abstraídos por essa função, mas, aos que se interessarem, recomenda-se a busca pelo tema de matrizes de rotação dentro da álgebra linear.

A seguir, o código completo e imagem de demonstração.

```
import cv2  
imgOrig = cv2.imread("mario.jpg")  
linhas, colunas = imgOrig.shape[:2]  
matriz = cv2.getRotationMatrix2D(  
(colunas / 2, linhas / 2), 90, 1)  
imgNova = cv2.warpAffine(  
imgOrig,matriz,(colunas, linhas))  
cv2.imshow("Resposta", imgNova)  
cv2.waitKey(0)
```

Saída do Programa:

Figura 7 – Imagem Original da esquerda e rotacionada na direita





3.2 Translação

A translação consiste em movimentar a imagem tanto no eixo horizontal quanto vertical. É semelhante à operação de rotação, devemos fazer o mesmo processo, primeiro gerar uma matriz de translação e, na sequência, aplicar ela na imagem com a função *warpAffine*.

A matriz de transformação bidimensional possui o formato representado a seguir.

$$\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \end{bmatrix}$$

Na matriz, o valor **tx** representa o deslocamento horizontal, enquanto o valor **ty** representa o deslocamento vertical. E, para criarmos uma matriz assim, podemos utilizar a função `float32` dentro da biblioteca `numpy`. Valores positivos de **tx** movem a imagem para a direita, e valores negativos para esquerda. Enquanto valores positivos de **ty** movem a imagem para baixo e valores negativos para cima. Com a seguinte linha de código, geramos uma matriz que movimenta a imagem 100 *pixel* para direita e 200 para baixo.

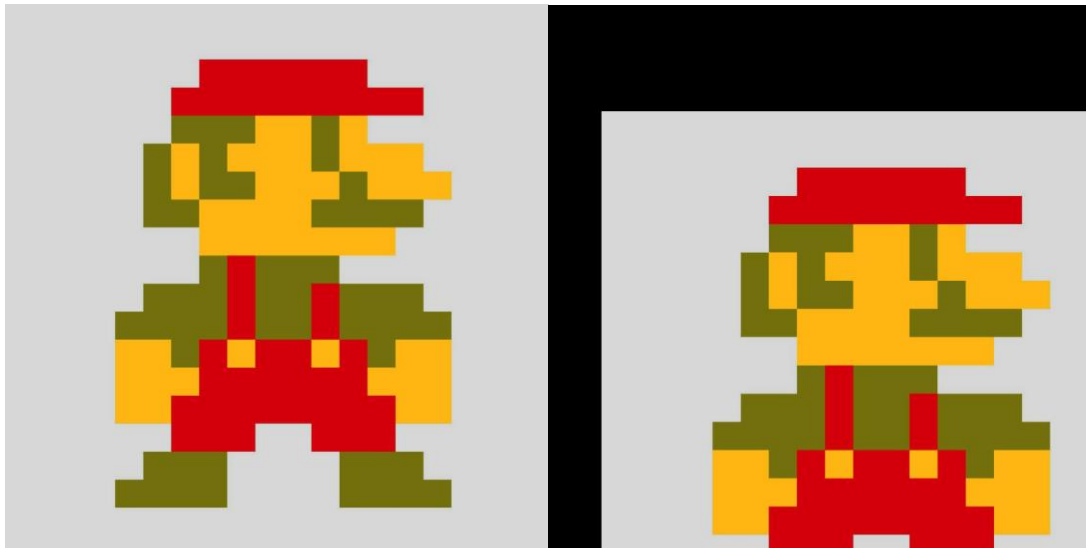
```
matriz = np.float32([[1, 0, 100], [0, 1, 200]])
```

Confira a seguir o código completo:

```
import cv2
import numpy as np
imgOrig = cv2.imread("mario.jpg")
linhas, colunas = imgOrig.shape[:2]
matriz = np.float32([[1, 0, 100], [0, 1, 200]])
imgNova = cv2.warpAffine(
imgOrig, matriz, (colunas, linhas))
cv2.imshow("Resposta", imgNova)
cv2.waitKey(0)
```




Figura 8 – Imagem Original da esquerda transladada à direita



Fonte: Gomes, 2021.

3.3 Escala

A operação de escala é utilizada para quando desejamos modificar as dimensões de uma imagem. Para isso, utilizamos no opencv a função *resize*. Ela pode ser executada de diversas formas, no entanto mostramos a mais básica dela.

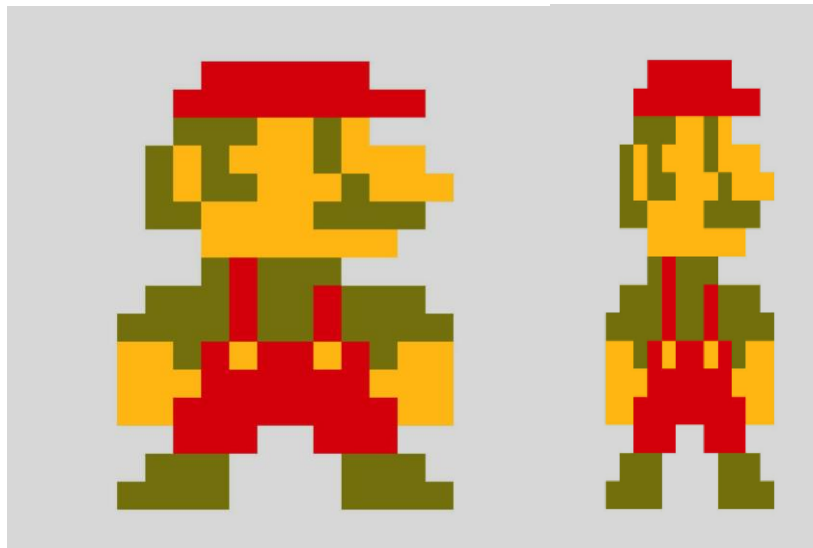
Primeiro parâmetro, a imagem original, no segundo parâmetro, uma tupla com as dimensões desejadas de largura e altura, respectivamente, em número de *pixels*. É possível passar mais parâmetros como especificar o algoritmo desejado para a interpolação, que é o método em si de diminuir ou aumentar as dimensões da imagem.

No código de exemplo a seguir, a imagem *mario.jpg* passa pela operação de escala e suas medidas se tornam 200 *pixels* de largura por 400.

```
import cv2
imgOrig = cv2.imread("mario.jpg")
imgNova = cv2.resize(imgOrig, (200,400))
cv2.imshow("Resposta", imgNova)
cv2.waitKey(0)
```



Figura 9 – Imagem Original da esquerda e com escala na direita



Fonte: Gomes, 2021.

3.4 Perspectiva

Por vezes, a região de interesse da imagem foi capturada de um ponto de vista que não é ideal, ou propriedades do conjunto de lentes adotada na câmera podem gerar distorções na imagem. Para essas situações, podemos aplicar algoritmos que corrigem a perspectiva da imagem.

No opencv, podemos gerar uma matriz de transformação utilizando a função *getPerspectiveTransform*. Ela recebe uma matriz com as coordenadas dos 4 pontos da região de interesse da imagem, e uma segunda matriz com os 4 pontos equivalentes no destino. Por exemplo, no código a seguir, uma placa de um carro, os pontos marcados são as coordenadas em *pixels* dos 4 cantos da placa e, no destino, marcamos uma região retangular na qual queremos gerar a resposta.

Na sequência, utilizamos a matriz gerada pela função anterior e aplicamos ela na imagem utilizando a função *warpPerspective*, que recebe como parâmetro a imagem original, a matriz de transformação e uma tupla com as dimensões da nova imagem. Confira o código e sua execução a seguir.

```
import cv2
import numpy as np
imgOrig = cv2.imread("placa.png")
```



```
pontosOrigem = np.float32([
    [13,1], [193,61], [13,51], [185,111]])
pontosDestino = np.float32([
    [0,0], [200,0], [0,50], [200,50]])
matriz = cv2.getPerspectiveTransform(
    pontosOrigem, pontosDestino)
imgNova = cv2.warpPerspective(
    imgOrig, matriz, (200, 50))
cv2.imshow("Resposta", imgNova)
cv2.waitKey(0)
```

Figura 10 – Imagem com correção de perspectiva de uma placa de carro



Crédito: Blue Corner Studio/Shutterstock.

TEMA 4 – OPERAÇÕES ARITMÉTICAS

Assim como podemos aplicar operações aritméticas em matrizes, podemos, da mesma forma, realizar operações aritméticas nas imagens. Soma, subtração, multiplicação etc. podem ser adotados para realçar ou combinar imagens.

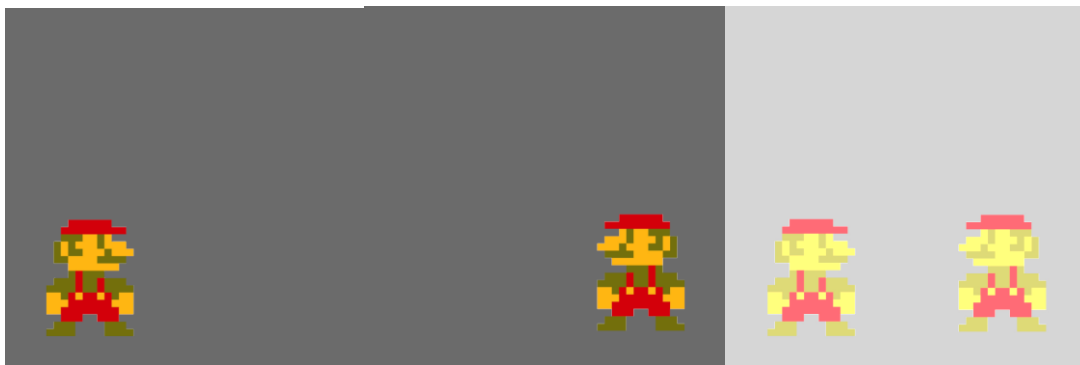
4.1 Adição

Por vezes, dependendo da aplicação, é interessante combinar duas imagens em uma só. E para isso o opencv conta com uma função muito simples chamada *add*, que recebe duas imagens distintas e as soma *pixel a pixel*. É importante que as duas imagens tenham o mesmo *bit depth* e tamanho.

Confira o código a seguir:

```
import cv2
imagem1 = cv2.imread("marioEsquerda.jpg")
imagem2 = cv2.imread("marioDireita.jpg")
imagemResultado = cv2.add(imagem1, imagem2)
cv2.imshow("Resposta", imagemResultado)
cv2.waitKey(0)
```

Figura 11 – Imagem da esquerda e do meio somadas na imagem resultante da direita



Fonte: Gomes, 2021.

A imagem do resultado ficou mais clara do que as originais pelo fundo ter sido somado. É possível também somar um valor fixo em todos os *pixels* da imagem se passarmos um valor fixo em vez de uma outra imagem, permitindo deixarmos a imagem mais clara ou mais escura, caso seja somado um valor negativo. Confira o código a seguir.

```
import cv2
imagem = cv2.imread("caracol.png", 0)
imagemResultado = cv2.add(imagem, 50)
cv2.imshow("Resposta", imagemResultado)
cv2.waitKey(0)
```



Figura 12 – Imagem original na esquerda e imagem com o valor 50 somado aos *pixels* na direita



Crédito: 4-life-2-b/Shutterstock.

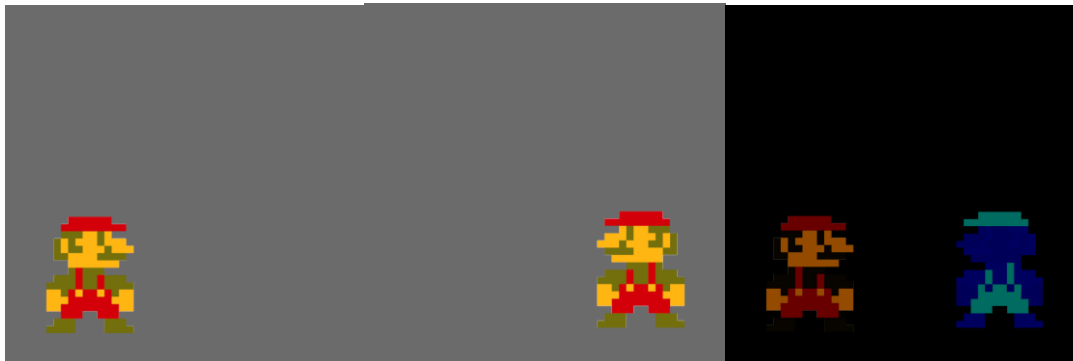
4.2 Subtração

É uma operação muito importante para identificar a diferença entre duas imagens. Suponha, por exemplo, uma câmera de segurança que faça a subtração da imagem capturada no momento atual com a que foi capturada instantes antes, o resultado será exatamente o recorte do que se moveu na cena. A função de subtração no opencv se chama *subtract* e recebe duas imagens. Confira o código a seguir.

```
import cv2
imagem1 = cv2.imread("marioEsquerda.jpg")
imagem2 = cv2.imread("marioDireita.jpg")
imagemResultado = cv2.subtract(imagem1, imagem2)
cv2.imshow("Resposta", imagemResultado)
cv2.waitKey(0)
```



Figura 13 – A imagem da direita é resultado da subtração da imagem da esquerda com a do centro



Fonte: Gomes, 2021.

Tudo o que for diferente do valor 0 na imagem resultado de uma subtração é o que tem de diferença nas imagens. No caso anterior, por exemplo, se fossem duas capturas feitas por uma câmera, poderíamos utilizar para calcular o quanto o personagem se moveu de uma imagem para outra.

4.3 Combinação

Outra operação aritmética possível com imagens é combinação, que significa mesclar duas imagens em uma só. Esse tipo de atividade é especialmente útil para quando desejamos, por exemplo, reduzir ruído de imagens combinando mais uma imagem capturada por um mesmo sensor de um mesmo ponto de vista. Falaremos mais sobre redução de ruído no próximo tema.

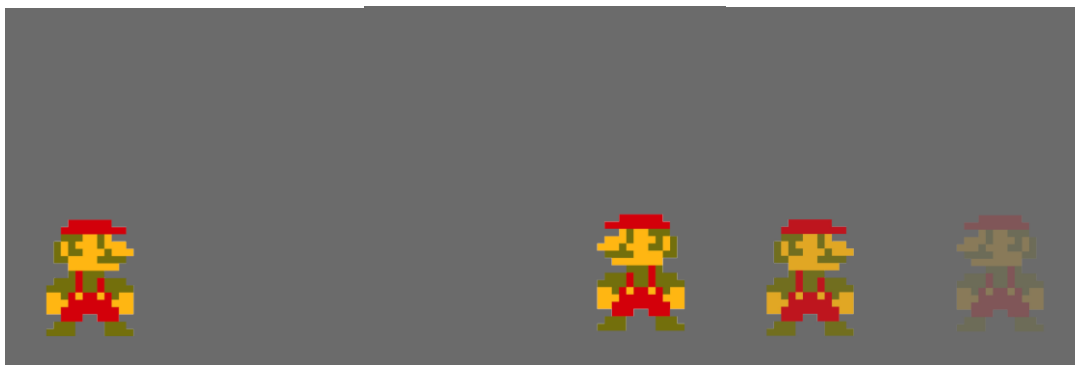
No *opencv*, a função que realiza essa mescla é *addWeighted*, ela possui como parâmetros as imagens que ela combina e, junto, um peso que deve ser associado. No código a seguir, por exemplo, temos que a primeira imagem representa 0.8 (80%) e a segunda imagem tem peso 0.2 (20%) da composição da imagem de resultado. Essa proporção de 80/20 é a razão da segunda imagem ser menos nítida.

Existe um quinto parâmetro na função *addWeighted* que consiste em um incremento na imagem final para torná-la mais clara, ou escura, caso o incremento seja um valor negativo. A seguir, você encontra o código e o resultado de sua execução.

```
import cv2
```

```
imagem1 = cv2.imread("marioDireita.jpg")
imagem2 = cv2.imread("marioEsquerda.jpg")
imagemResultado = cv2.addWeighted(
imagem1, 0.2, imagem2, 0.8, 0)
cv2.imshow("Resultado", imagemResultado)
cv2.waitKey(0)
```

Figura 14 – A imagem da direita é resultado da combinação da imagem da esquerda com a do centro



Fonte: Gomes, 2021.

TEMA 5 – RUÍDOS EM IMAGENS

O ruído de imagem é uma interferência na leitura dos dados que gera algum tipo variação aleatória e inesperada na leitura da cor ou luminosidade. O termo surgiu com a transmissão de sinais de rádio e suas flutuações elétricas que geram estática, que também é chamado de ruído.

Para a visão computacional e processamento de imagens, o ruído pode comprometer significativamente os resultados. Para melhor entender esse problema e posteriormente as formas de atenuá-lo, vamos discutir na sequência os diferentes tipos de ruídos presente nas imagens digitais.

5.1 Ruído gaussiano

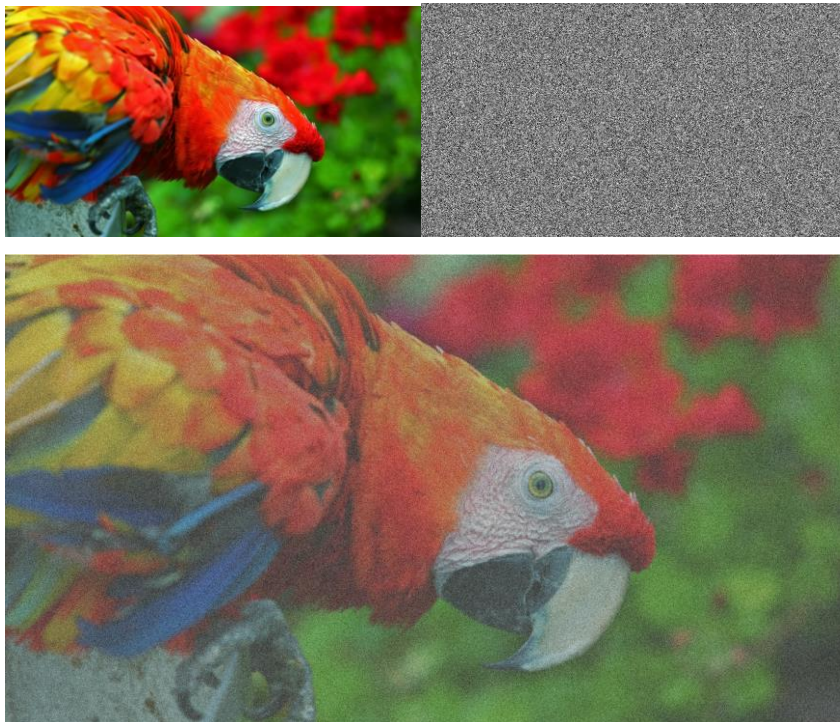
A causa principal desse tipo de ruído em imagens digitais são problemas no momento da captura, como iluminação precária ou elevada temperatura do sensor, ou mesmo durante a transmissão da imagem dentro do circuito.



Esse ruído lembra uma estática e possui uma distribuição de probabilidade normal ao longo da imagem. Pode ser corrigido com o uso de filtros espaciais com o custo de apresentar certo grau de borramento. Veremos mais sobre filtros futuramente.

A seguir, temos um exemplo da imagem original, o ruído gerado e o resultado da adição do ruído gaussiano na imagem original.

Figura 15 – Imagem original, ruído gaussiano e imagem somada ao ruído



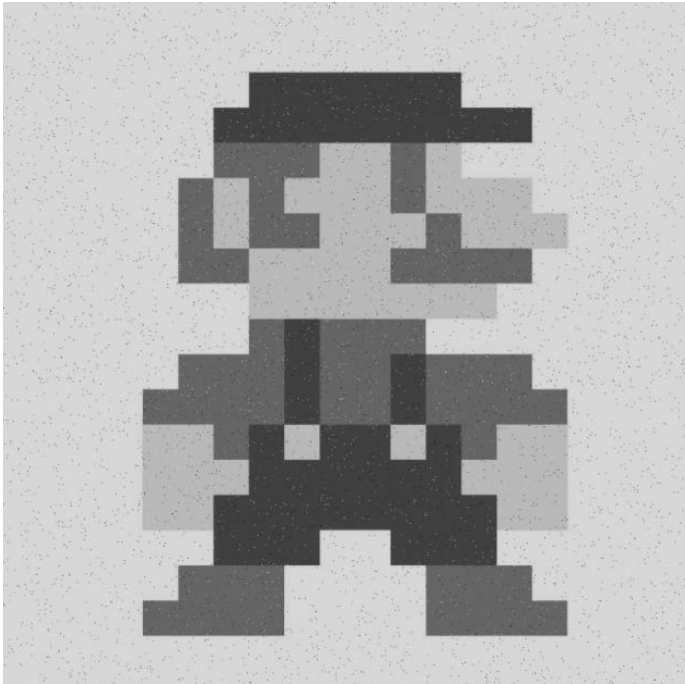
Crédito: Goran Jakus/Shutterstock.

5.2 Ruído tipo sal e pimenta

Esse ruído é caracterizado por pontos pretos(pimenta) em regiões claras e pontos claros(sal) em regiões escuras, como se fossem espalhados de forma esparsa na imagem. Esse tipo de problema é comum ocorrer durante a conversão de imagem analógica para digital e erros de conversão de *bit* durante a transmissão etc. Novamente, pode ser atenuado com filtros espaciais ou com a combinação de imagens sucessivas capturadas pelo mesmo sensor, visto que o padrão desse tipo de ruído tende a ser aleatório.



Figura 16 – Imagem com ruído sal e pimenta

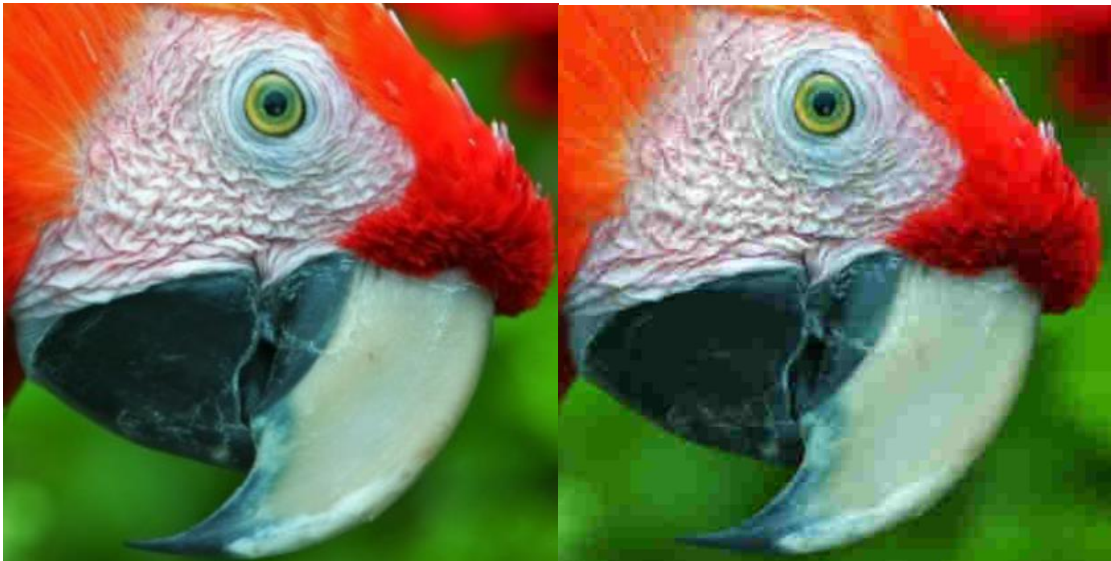


Fonte: Gomes, 2021.

5.3 Ruído por codificação de imagem

Esse tipo de ruído ocorre durante a conversão de imagens para formatos compactados com perda de dados, como o jpg. Quando configurado para um elevado grau de compactação, ocorre perda de informação e gera um aspecto borrado quadriculado na imagem. A seguir, temos um exemplo desse efeito.

Figura 17 – Imagem original na esquerda e afetada pela ruído da compressão jpg na direita



Crédito: Goran Jakus/Shutterstock.

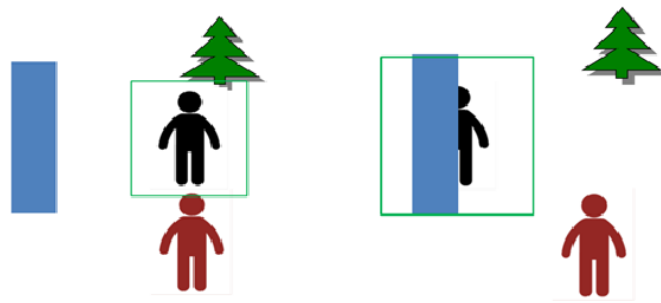
5.4 Oclusões na imagem

Não é exatamente um ruído de sinal, mas outro problema que acontece durante a aquisição da imagem são oclusões, objetos indesejados obstruindo a visualização da região de interesse. Com o uso de técnicas temporais que combinam a informação de uma sequência de imagens de uma mesma cena, é possível atenuar esse efeito para certas aplicações. Sensores que capturam a profundidade de cada ponto da cena podem utilizar essa informação também para remover qualquer *pixel* com um valor de profundidade fora do desejado.

Na imagem a seguir, é ilustrado o problema de oclusão durante o rastreamento de pessoas em vídeo. Um objeto que oculte a pessoa por alguns *frames* do vídeo é um desafio adicional para essa tarefa.



Figura 18 – Ilustração representando a oclusão durante o rastreo de pessoas



Fonte: Imagem elaborada com base em Hiromasa Takada, 2015.

FINALIZANDO

Nesta aula, discutimos os principais problemas enfrentados na etapa de pré-processamento. O acesso aos *pixels*, histogramas, transformações geométricas, entre outros, oferece um ferramental básico para manipularmos e entendermos as imagens para melhor se adaptarem às nossas necessidades nos algoritmos de visão computacional. Futuramente, vamos continuar discutindo formas de manipulação da imagem com filtros e detectores de borda.



REFERÊNCIAS

- ANDALÓ, F. **Modelagem e Animação 2D e 3D para Jogos**. Érica, 2015.
- ARRUDA, E. P. **Fundamentos para o Desenvolvimento de Jogos Digitais**: Bookman, 2014. (Série Tekne)
- BANIN, S. **Python 3**: conceitos e aplicações: uma abordagem. 1. ed. Erica, 2018.
- BARELLI, F. **Introdução à visão computacional**. 1. ed. Casa do Código, 2019.
- CHONG, A. **Animação digital**: coleção animação básica. AMGH, 2014.
- FRIGERI, S. **Computação Gráfica**. 1. ed. SAGAH, 2018.
- GONZALEZ, R. C.; WOODS, R. E. **Processamento Digital de Imagens**. 3. ed. São Paulo: Prentice Hall, 2010.
- SANTOS, F. dos, FERREIRA, S. F. **Geometria Analítica**. ArtMed, 2009.
- WINTERLE, P. **Geometria Analítica**. 2. ed. São Paulo: Pearson, 2014.
-